

Министерство науки и высшего образования РФ
ФГБОУ ВО «Нижневартковский государственный университет»
Факультет информационных технологий и математики
Кафедра информатики и методики преподавания информатики

Курсовой проект по дисциплине
«Основы научно-исследовательской деятельности»

МЕТОДЫ ИНТЕЛЛЕКТУАЛЬНОГО АНАЛИЗА СТРАХОВЫХ ВЫПЛАТ

Исполнитель:
студент группы 3353
Горидько Иван Александрович
Руководитель:
кандидат технических наук,
доцент кафедры информатики и
методики
преподавания информатики
Катермина Татьяна Сергеевна

(подпись)

Нижневартковск, 2026

Commented [U1]: вы переехали в новую группу и теперь ваша тема звучит так: «Методы интеллектуального анализа страховых выплат»
На содержание это не повлияет, но тему поменяйте

Оглавление

Глава 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПРОГНОЗИРОВАНИЯ СТРАХОВЫХ ВЫПЛАТ СРЕДСТВАМИ МАШИННОГО ОБУЧЕНИЯ	5
1.1. Задача предсказания страховых выплат	5
1.2. Уточненная постановка задачи	5
1.3. Библиотеки, язык и среда разработки	6
1.4. Анализ существующих методов предсказания страховых выплат.....	8
1.4.1. Линейная Регрессия.....	8
1.4.2. Множественная линейная регрессия	11
1.4.3. Регуляризация	12
1.4.4. Настройка гиперпараметров	17
1.4.5. Деревья решений.....	19
1.4.6. Регрессия Случайным Лесом	19
1.4.7. Градиентный бустинг	22
ГЛАВА 2. ПРАКТИЧЕСКОЕ ПРИМЕНЕНИЕ СРЕДСТВ МАШИННОГО ОБУЧЕНИЯ ДЛЯ ПРОГНОЗИРОВАНИЯ СТРАХОВЫХ ВЫПЛАТ.....	25
2.1. Анализ исходного датасета. Предобработка данных, кодирование признаков и извлечение признаков. Первичный разведочный анализ данных (EDA). Удаление выбросов.	25
2.2. Обоснование выбора моделей машинного обучения. Анализ результатов обучения. Описание метрик MAE, RMSE, R ²	34
2.3. Вывод. Выбор лучшей модели	37
Список используемых источников.....	39
Приложения.....	41
Приложение 1.....	41
Приложение 2.....	48

Commented [U2]: В оглавление попало двоеточие после списка используемых источников, уберите

Анализ статистики страховых выплат средствами языка Python

Введение

В современном мире страхование является неотъемлемой частью финансовой стабильности как отдельных людей, так и целых предприятий. Прогнозирование размера медицинских страховых выплат на основе статистических данных о клиентах (возраст, пол, ИМТ, курение, регион) с использованием методов машинного обучения позволяет повысить точность оценки рисков и обоснованность тарифов в системе здравоохранения.

Объект исследования: формирование размеров страховых выплат на основе статистических данных.

Процесс исследования: формирование размеров страховых выплат средствами интеллектуального анализа данных.

Цель исследования: реализовать предсказания страховых выплат средствами языка python.

Задачи исследования:

- Провести анализ и предобработку предоставленного набора данных о страховых случаях.
- Реализовать и обучить модель линейной регрессии для прогнозирования размера страховых выплат.
- Реализовать и обучить модель линейной регрессии с применением алгоритмов “Регуляризации”.
- Провести оптимизацию гиперпараметров для обучения.
- Реализовать и обучить модель регрессии на основе алгоритма “Случайный лес”.
- Реализовать и обучить модель регрессии на основе алгоритма “Градиентный бустинг”.

- Провести сравнительный анализ эффективности построенных моделей и выбрать оптимальную.

Практическая значимость исследования: Разработанные модели машинного обучения позволяют прогнозировать размер страховых выплат на основе индивидуальных данных клиента (возраст, ИМТ, курение и др.). Полученные результаты могут использоваться страховыми компаниями для корректировки тарифов, выявления факторов риска и повышения финансовой эффективности.

Commented [U3]: Здесь коротко сформулируйте Практическую значимость исследования

Глава 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПРОГНОЗИРОВАНИЯ

СТРАХОВЫХ ВЫПЛАТ СРЕДСТВАМИ МАШИННОГО ОБУЧЕНИЯ

Commented [U4]: я скидывала в облако методичку по курсовые. Посмотрите там форматирование (отступы, интервалы и т.д.)

1.1. Задача предсказания страховых выплат

Задача предсказания [1] страховых выплат относится к классу задач прогнозной аналитики и машинного обучения с учителем, а именно — к задачам регрессии.

Её основная цель заключается в построении математической модели, способной на основе исторических данных о клиентах (таких как возраст, индекс массы тела, вредные привычки, регион проживания и другие демографические признаки) прогнозировать непрерывную числовую величину — ожидаемый размер страховой выплаты.

Точное прогнозирование позволяет страховым компаниям оптимизировать тарифную политику, более эффективно управлять рисками и финансовыми резервами, а также выявлять скрытые закономерности, влияющие на стоимость страховых случаев.

1.2. Уточненная постановка задачи

Для решения поставленной задачи должны быть выполнены следующие этапы:

1. Выбрать исходный набор данных о медицинских страховых выплатах.
2. Провести предобработку данных: нормализовать числовые признаки, обработать категориальные переменные с помощью One-Hot

Commented [U5]: в первой главе вы не пишете, что у вас там «делается», а пишете, что «должно быть сделано». Ставите задачу. Можно прямо перечислением:
1. Выбрать набор данных.
2. Предобработать набор данных – нормализовать, избавиться от пропусков, выбросов, проверить и преобразовать типы данных.
3. Проверить корреляцию между признаками, построить тепловую карту корреляции.
4. Отбросить признаки... по причинам...
5. Провести анализ... методами...
6. Определить и проанализировать полученные метрики.....
7. Выбрать наилучшие алгоритмы....., обосновать выбор...

Encoding, удалить выбросы по методу межквартильного размаха (IQR).

3. Проверить корреляцию между признаками.
4. Отбросить признаки, создающие мультиколлинеарность (например, один из дамми-признаков для региона и весовой категории).
5. Провести разведочный анализ данных (EDA) с визуализацией распределений и взаимосвязей.
6. Реализовать и обучить модели: линейную регрессию, гребневую регрессию (Ridge), Лассо (Lasso), случайный лес (Random Forest), градиентный бустинг (Gradient Boosting).
7. Выполнить настройку гиперпараметров с помощью сеточного поиска (GridSearchCV) и кросс-валидации.
8. Оценить качество моделей по метрикам MAE, RMSE, R^2 .
9. Выбрать наилучшую модель на основе сравнения полученных метрик, обосновать выбор.

1.3. Библиотеки, язык и среда разработки

Для реализации поставленных задач в качестве языка программирования следует выбрать Python версии 3.x. Этот выбор обусловлен его широкой распространённостью в области анализа данных и машинного обучения, наличием большого количества специализированных библиотек с открытым исходным кодом, а также активным сообществом, что облегчает поиск решений для типовых задач.

В качестве среды разработки целесообразно использовать Jupyter Notebook, поскольку она позволяет сочетать исполняемый код, визуализацию результатов и текстовые пояснения в одном документе, что удобно для исследовательских проектов.

Для загрузки и манипуляции данными необходимо применить библиотеку Pandas, предоставляющую удобные структуры данных (DataFrame) и функции для фильтрации, группировки и преобразования. Для численных расчётов — библиотеку NumPy. Для визуализации — Matplotlib и Seaborn, которые позволяют строить гистограммы, ящики с усами, диаграммы рассеяния и тепловые карты корреляции.

Реализация алгоритмов машинного обучения (линейная регрессия, регуляризация, случайный лес, градиентный бустинг) лучше всего будет выполнена с помощью библиотеки Scikit-learn, которая содержит эффективные и проверенные реализации этих методов, а также инструменты для предобработки данных (StandardScaler), разбиения выборки (train_test_split), настройки гиперпараметров (GridSearchCV) и расчёта метрик качества (MAE, RMSE, R^2).

1.4. Анализ существующих методов предсказания страховых выплат.

1.4.1. Линейная Регрессия

Первоначальная цель любого метода – создать модель, которая объясняет закономерности для каких-то данных из реального мира, так, чтобы стало возможным предсказать что может произойти, с различными вводными данными.

Простейшая модель, которой можно аппроксимировать данные – это линия. При попытке поиска линии, которая лучше всего соответствует набору данных – используется Линейная Регрессия [2, 3].

Например:

- Рыночная цена дома в зависимости от его площади. Можно ли предсказать за сколько будет продан дом, учитывая его площадь?
- Налоговая ставка страны в зависимости от ее ВВП. Можно ли прогнозировать налогообложение на основе ВВП страны?

Прямая определяется своим наклоном и точкой пересечения. Это можно выразить через формулу:

$$y = mx + b,$$

где m – наклон, b – точка пересечения. y заданная точка на оси- y , которая соответствует заданному x на оси- x .

Для каждой точки высчитывается функция потерь — это число, которое измеряет неточность предсказания модели.

Commented [U6]: укажите, какая функция ошибки используется

В качестве функции потерь (убытка) в линейной регрессии [2, 3] обычно используется среднеквадратичная ошибка (MSE — Mean Squared Error), которая вычисляется как среднее арифметическое квадратов разностей между фактическими и предсказанными значениями:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Минимизация MSE приводит к нахождению оптимальных коэффициентов модели.»

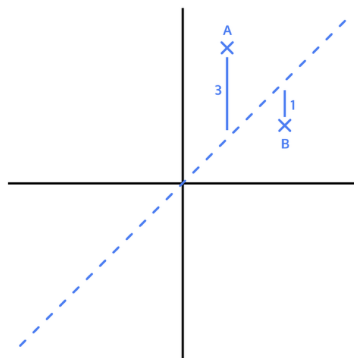


Рис. 1.1. Вычисление убытка

- для точки A, дистанция в квадрате: 9.
- для точки B, расстояние в квадрате: 1.

Функция потерь в этой модели равна 10. Если найти линию, функция потерь которой меньше 10 – эта линия станет более подходящей моделью для данных.

Минимизируя функцию потерь, специалист по машинному обучению продвигает каждый параметр вниз по кривой до того момента, пока он уменьшает функцию потерь и останавливается в тот момент, когда достигается минимальное значение. Этот процесс называется градиентным спуском [2, 3].

Градиентом называется спуск кривой в любой точке. Градиенты вычисляются по формуле:

Commented [U8]: вы всю эту теорию не сами придумали, значит должны быть ссылки на источники. Они указываются в квадратных скобках вот так [4], где 4 – номер источника в списке литературы.

Commented [ИГ8R2]: Это сделать невозможно. От ссылки на CodeCademy Вы отказались!

$$-2N \sum_{i=1}^N (y_i - (mx_i + b)),$$

где:

- N – количество точек в наборе данных,
- m – текущее предположение о градиенте,
- b – текущее предположение о точке пересечения.

Чтобы найти m , или изменение убытка при изменении кривой, используется формула:

$$-2N \sum_{i=1}^N x_i (y_i - (mx_i + b))$$

Для того, чтобы масштабировать размер шага, нужно умножить градиент на параметр скорости обучения.

$$\text{новое_}b = \text{текущее_}b - (\text{скорость_обучения} \times b_ \text{градиент}),$$

Где новое_ b – это предположение относительно значения b , $b_ \text{градиент}$ – это градиент кривой потерь при текущем предположении, а переменная скорость_обучения – пропорциональна размеру шага, который нужно сделать.

Чтобы узнать когда остановить обучение модели, необходимо определить сходимость. **Сходимость** – это когда функция потерь перестает изменяться (или изменяется очень медленно)

Необходимо выбрать параметр скорости обучения, которая будет определять продвижение вниз по кривой. Слишком низкий параметр скорости обучения потребует много времени для достижения сходимости. Слишком высокий параметр скорости обучения может пропустить наилучшее значение. В итоге возможно так и не достигнуть сходимости.

Для обучения модели не требуется поиск абсолютно наилучшей скорости обучения. Достаточно найти такой параметр скорости обучения, чтобы градиентный спуск сходился с необходимой эффективностью.

1.4.2. Множественная линейная регрессия

Множественная линейная [4] регрессия использует две или более независимых переменных для прогнозирования значений зависимой переменной. Она основана на следующем уравнении:

$$y = b + m_1x_1 + m_2x_2 + \dots + m_nx_n.$$

Здесь $m_1, m_2, m_3, \dots, m_n$ — это коэффициенты, а b — это свободный член (интерсепт), которые необходимо найти.

1.4.3. Регуляризация

Регуляризация [3, 4] играет очень важную роль в практической реализации моделей машинного обучения. Это статистический метод, который минимизирует переобучение и применяется на этапе подгонки модели. Регуляризация также относится к встроенным методам отбора признаков, поскольку она выполняется в процессе расчёта параметров модели.

Алгоритмы машинного обучения редко внедряются в промышленную эксплуатацию без использования какой-либо формы регуляризации. Причина этого заключается в следующем: на практике каждая модель должна решать вопрос о том, насколько хорошо она способна обобщать известные данные на неизвестные. Мы можем обучать, тестировать и настраивать модели на известных данных, добиваясь максимальной точности. Однако при внедрении моделей мы применяем их к новым данным. Регуляризация гарантирует, что наша модель остаётся точной.

Если модель хорошо воспроизводит конкретный набор точек данных, но плохо обобщается на новые данные, она переобучена. Такая модель обладает одним или несколькими из следующих свойств:

- Хорошо подгоняется к обучающим данным, но показывает значительно худшие результаты на тестовых данных.
- Обычно имеет больше параметров, чем необходимо, то есть обладает высокой сложностью модели.
- Может подгоняться под мультиколлинеарные признаки (то есть признаки, которые имеют высокую отрицательную или положительную корреляцию).
- Может подстраиваться под шум в данных и, вероятно, ошибочно принимать шум за значимые признаки.

Регуляризация налагает штраф на переобучение путём добавления «штрафного члена» к функции потерь. Два наиболее распространённых способа реализации этой идеи — регуляризация “Лассо (L1)” и “ридж-регуляризация (L2)”. Оба метода ограничивают переобучение за счёт контроля величины коэффициентов модели. Штрафной (регуляризационный) член умножается на коэффициент альфа и прибавляется к исходной функции потерь:

$$\text{Новая функция потерь} = \text{Старая функция потерь} + \alpha \times \text{Регуляризационный член}$$

Из-за аддитивного члена минимизация новой функции потерь неизбежно приводит к увеличению «истинных потерь» — то есть показатели качества на обучающих данных будут хуже, чем при регрессии без регуляризации. Но именно и добивается специалист по машинному обучению, применяя регуляризацию, так как его модель переобучена (она хорошо работает на обучении, но плохо обобщается на тестовых данных).

Регуляризационный член представляет собой сумму абсолютных значений коэффициентов в случае L1 и сумму квадратов коэффициентов в случае L2:

$$L1 = |b_1| + |b_2|$$

$$L2 = b_1^2 + b_2^2$$

В сценарии линейной регрессии с двумя признаками, новая функция потерь для L1 имеет следующий вид:

$$L1 \text{ потерь} = \frac{1}{n} \sum_{i=1}^n (y_i - b_0 - b_1 x_{1i} - b_2 x_{2i})^2 + \alpha \times (|b_1| + |b_2|)$$

Минимизация этой функции означает ограничение величины регуляризационного члена при одновременной минимизации исходной функции потерь. Пусть регуляризационный член может принимать максимальное значение s :

$$|b_1| + |b_2| \leq s$$

Это эквивалентно ограничению коэффициентов областью вокруг начала координат, ограниченной четырьмя прямыми:

$$b_1 + b_2 = s; b_1 - b_2 = s; b_2 - b_1 = s; -b_1 - b_2 = s;$$

Уже было рассмотрено как L1-регуляризация уменьшает переобучение и одновременно выступает в роли метода отбора признаков, на примере двух признаков. Но как это обобщается на набор данных с множеством признаков? Функция потерь для множественной линейной регрессии с m признаками выглядит так:

$$L1 \text{ потеря} = \frac{1}{n} \sum_{i=1}^n (y_i - b_0 - b_1 x_{1i} - b_2 x_{2i} - \dots - b_m x_{mi})^2 + \alpha \times (|b_1| + |b_2| + \dots + |b_m|)$$

L2-регуляризацию также называют ридж-регуляризацией, поскольку её математическая формулировка относится к классу методов «ридж-анализа». Модифицированная функция потерь для L2 выглядит следующим образом:

$$L2 \text{ потеря} = \frac{1}{n} \sum_{i=1}^n (y_i - b_0 - b_1 x_{1i} - b_2 x_{2i})^2 + \alpha \times (b_1^2 + b_2^2)$$

Аналогично L1, это означает ограничение регуляризационного члена поверхностью:

$$b_1^2 + b_2^2 \leq s^2$$

Слово Lasso — это аббревиатура от «Least Absolute Shrinkage and Selection Operator» (оператор сжатия и отбора наименьших абсолютных значений). Он сжимает абсолютные значения коэффициентов и осуществляет отбор параметров, обнуляя некоторые из них. Если коэффициент регрессионной модели обнулён, это означает, что соответствующий признак практически исключён. В моделях с большим числом признаков L1-регуляризация обычно обнуляет значительную долю параметров, выполняя тем самым функцию отбора признаков.

В отличие от Lasso, коэффициенты Ridge никогда не становятся точно равными нулю. Таким образом, L2-регуляризация не является методом исключения признаков в отличие от L1. Коэффициенты L2 могут быть сколь

угодно малыми, но не нулевыми. Это особенно полезно, когда нет необходимости полностью отбрасывать признаки, но при этом необходимо подчеркнуть их относительную значимость.

Функция потерь для множественной линейной регрессии с L2-регуляризацией и m признаками имеет вид:

$$L2 \text{ потеря} = \frac{1}{n} \sum_{i=1}^n (y_i - b_0 - b_1 x_{1i} - b_2 x_{2i} - \dots - b_m x_{mi})^2 + \alpha \times (b_1^2 + b_2^2 + \dots + b_m^2)$$

Альфа — это так называемый гиперпараметр в машинном обучении. Он не обучается на этапе подгонки модели, как параметры модели. Вместо этого он выбирается до начала обучения и используется для управления процессом обучения. В регуляризации значение альфа определяет, насколько сильно специалист по машинному обучению хочет контролировать переобучение на этапе подгонки модели.

Альфа позволяет контролировать размер области ограничения коэффициентов и, соответственно, величину самих коэффициентов. Чем больше альфа, тем меньше допустимые значения коэффициентов. По сути, альфа обратно пропорциональна s в случае L1 или s^2 в случае L2.

Необходимо рассмотреть все возможные варианты размеров границ ограничений для обоих типов регуляризации.

- Если s очень велико (то есть альфа очень мало), минимум исходной функции потерь легко попадает внутрь этой большой области, и результат становится похожим на регрессию без регуляризации.
- Если s очень мало (то есть альфа очень велико), коэффициенты регрессии становятся очень маленькими, а значение функции потерь для наилучшей подгонки — большим, что приводит к избыточной регуляризации.

Чтобы избежать этих крайних случаев, необходимо найти правильную степень регуляризации путём настройки альфа.

Добавляя регуляризационный член к функции потерь, специалист по машинному обучению вносит в задачу смещение (bias), то есть подталкивает

модель к тому, чтобы коэффициенты оставались внутри границы регуляризации. Чем больше альфа, тем меньше коэффициенты и тем более смещённой становится модель.

При очень больших значениях альфа Ridge делает большинство коэффициентов очень маленькими, тогда как Lasso сводит к нулю все коэффициенты, кроме одного.

Регуляризация применяется для предотвращения переобучения. Переобученная модель обладает высокой дисперсией (variance), поскольку она может подстраиваться под случайные ошибки или шум. Введение смещения призвано уменьшить дисперсию, но если не контролировать его и допустить чрезмерное увеличение, возникает риск недообучения.

В идеале модель машинного обучения должна иметь низкое смещение и низкую дисперсию — то есть хорошо работать как на обучающих, так и на тестовых данных. Однако одновременное снижение смещения и дисперсии представляет собой проблему, так как уменьшение одного обычно ведёт к увеличению другого. Эта дилемма известна как компромисс смещения и дисперсии.

Настройка гиперпараметров [6, 11] помогает найти «золотую середину» в этом компромиссе, гарантируя, что ни смещение, ни дисперсия не станут слишком высокими. Обычно для этого выделяется часть данных, называемая валидационным набором (или holdout-набором) в дополнение к стандартному разделению на обучающую и тестовую выборки. Именно на валидационном наборе и выполняется настройка гиперпараметров.

1.4.4. Настройка гиперпараметров

Гиперпараметр модели машинного обучения — это величина, определяющая часть процесса обучения и не изменяемая в процессе тренировки, в отличие от параметра, который обучается непосредственно на данных.

Гиперпараметры могут как «сделать», так и «сломать» модель машинного обучения. Для понимания, необходимо рассмотреть алгоритм k-ближайших соседей. В классификации методом k-NN класс образца определяется классами его k-ближайших соседей из обучающей выборки. Если k слишком велико, классы с небольшим числом образцов могут быть пропущены, и модель может недообучиться. Если k слишком мало, выбросы в обучающих данных могут привести к неверной классификации соседей — модель переобучится на этих выбросах.

Этот баланс между переобучением и недообучением называется компромиссом смещения и дисперсии. Это важное понятие для обучения с учителем.

Смещение (bias) — разница между предсказаниями модели и истинными значениями. Модели с высоким смещением недообучены и показывают плохие результаты как на обучающей, так и на тестовой выборке. В таких моделях структура модели «пересиливает» обучающие данные.

Дисперсия (variance) — зависимость модели от обучающих данных. Модель обладает высокой дисперсией, если разные обучающие выборки приводят к сильно отличающимся результатам. Это часто ведёт к переобучению модели на обучающих данных. Переобученные модели с высокой дисперсией обычно хорошо работают на обучающих данных, но плохо на тестовых. В этом случае обучающие данные «пересиливают» структуру модели.

Компромисс смещения и дисперсии называется компромиссом, потому что уменьшение смещения обычно увеличивает дисперсию, и наоборот. Хорошие модели машинного обучения достигают баланса. Специалист по машинному обучению может уменьшить смещение, сделав модель более чувствительной к обучающим данным, но при этом рискует переобучить её. С другой стороны,

можно уменьшить дисперсию, ограничив модель так, чтобы она меньше зависела от обучающих данных, но тогда возрастает риск недообучения.

В конечном счёте необходимы модели, которые хорошо работают на тестовых данных. Один из способов — правильно выбрать гиперпараметры. В обучении с учителем нужны гиперпараметры, обеспечивающие хороший компромисс между смещением и дисперсией. Этого можно достичь, обучая модель многократно с разными значениями гиперпараметров и выбирая наилучшие. Этот процесс называется настройкой гиперпараметров (hyperparameter tuning). Существует несколько различных методов.

- **Алгоритм сеточного поиска** (grid search) для настройки гиперпараметров работает путём обучения модели на заранее заданных списках значений гиперпараметров. Метод перебирает все значения из списка и выбирает те, при которых модель показывает наилучший результат.
- **Алгоритм случайного поиска** (random search) работает аналогично, но вместо использования предопределённого списка значений гиперпараметров значения выбираются случайным образом. Как и при сеточном поиске, выбирается гиперпараметр, показавший наилучшую производительность.
- **Байесовская оптимизация** — ещё один подход к настройке гиперпараметров. Она использует идеи байесовской статистики для итеративного перебора различных значений гиперпараметров. Каждый раз, когда алгоритм байесовской оптимизации оценивает новое значение гиперпараметра, он получает больше информации о том, где следует искать оптимальное значение.
- **Генетические алгоритмы** также могут использоваться для настройки гиперпараметров. Они работают через несколько поколений значений гиперпараметров. Внутри каждого поколения наиболее приспособленные (то есть наилучшие по производительности) значения

гиперпараметров слегка мутируют (изменяются), чтобы произвести следующее поколение.

1.4.5. Деревья решений

Деревья решений [1, 5] — это модели машинного обучения с учителем, то есть они создаются на основе обучающего набора размеченных данных. Именно на этапе создания дерева и происходит «обучение» в машинном обучении.

Для лучшего понимания, необходимо смоделировать ситуацию: существует произвольное дерево, которое пытается предсказать получит ли студент отметку «А» на следующем экзамене или нет.

В начале все точки обучающего набора находятся на вершине дерева. Эти точки имеют метки: красные точки обозначают студентов, не получивших «А» на экзамене, а зелёные — студентов, получивших «А».

Затем дерево принимает решение разделить данные на меньшие группы на основе некоторого признака. Например, этим признаком может быть средняя оценка студента за курс. Студенты со средней оценкой «А» попадают в одну группу, со средней оценкой «В» — в другую и так далее.

После того как группы сформированы, процесс повторяется: дерево снова разделяет данные в каждой подгруппе, но уже по другому признаку. В конце концов оно достигает точки, когда дальнейшее разделение прекращается — это лист дерева. Теперь можно подсчитать метки данных в этом листе. Если неразмеченная точка попадает в этот лист, она классифицируется в соответствии с меткой большинства.

1.4.6. Регрессия Случайным Лесом

Деревья решений могут быть мощными моделями машинного обучения с учителем. Однако у них есть и недостатки: деревья решений часто склонны к переобучению. В прикладном мире существуют некоторые стратегии минимизации этой проблемы, например, метод отсечения ветвей (pruning) но

иногда этого недостаточно. Нужен другой способ обобщения деревьев. Здесь и пригодится понятие случайного леса.

Случайный лес [5] — это ансамблевая техника машинного обучения. Случайный лес содержит множество деревьев решений, которые совместно классифицируют новые точки. Когда случайному лесу предлагают классифицировать новую точку, лес передаёт эту точку каждому из деревьев решений. Каждое дерево сообщает свой классификационный результат, а случайный лес возвращает наиболее популярный класс. Подобно голосованию: каждое дерево имеет голос, и побеждает самый популярный класс. Некоторые деревья в случайном лесу могут быть переобучены, но поскольку предсказание основано на большом количестве деревьев, влияние переобучения снижается.

Для создания случайного леса используется техника агрегирования результатов (bagging) — сокращение от bootstrap aggregating (агрегирование бутстреп-выборок). Бутстреп — это метод выборки с возвращением.

Процесс работает так: каждый раз при создании дерева решений оно строится на основе другого подмножества точек из обучающего набора. Например, если в обучающем наборе 1000 строк, можно построить дерево, случайно выбрав 100 из этих строк. Таким образом, каждое дерево уникально, но все они созданы на части обучающих данных.

При бутстрепе выборка выполняется с возвращением. Для понимания данного принципа имеет место быть следующей визуализации: все 1000 строк помещены в мешок, и специалист по машинному обучению случайным образом вытаскивает одну строку. Записав, какая строка была выбрана, он возвращает её обратно. Это означает, что при выборе 1000 случайных строк одна и та же строка может быть выбрана несколько раз. Более того, хотя это маловероятно, все 1000 выбранных строк могут оказаться одинаковыми.

Поскольку выборка идёт с возвращением, нет необходимости сокращать размер обучающего набора с 1000 до 100. Специалист по машинному обучению может выбрать 1000 строк случайным образом, и из-за возможности

многократного выбора одной и той же строки всё равно получит уникальный набор данных.

Случайные леса создают различные деревья с помощью процесса, известного как агрегирование результатов (bagging), агрегирование бутстреп-выборок). Процесс начинается с построения одного дерева решений на бутстреп-выборке точек из обучающего набора. После того как множество деревьев создано, результаты «агрегируются». Для задач классификации агрегация часто представляет собой агрегирование результатов (bagging) большинством индивидуальных классификаторов. Для задач регрессии агрегация обычно заключается в усреднении предсказаний индивидуальных регрессоров.

В дополнение к использованию бутстреп-выборок из набора данных можно добавить разнообразия в способы построения деревьев, случайным образом отбирая используемые признаки.

При использовании обычного дерева решений используются все признаки, а разделение выбирается так, чтобы максимизировать прирост информации (information gain). Даже если это может показаться нелогичным, случайный выбор подмножества признаков может улучшить производительность ансамблевой модели.

В итоге, хоть отдельное дерево и может показывать худшие результаты, иногда увеличение дисперсии помогает общей производительности ансамблевой модели в целом.

1.4.7. Градиентный бустинг.

Бустинг — это последовательная техника обучения, в которой каждая базовая модель строится на основе предыдущей. Каждая последующая модель направлена на улучшение итоговой ансамблевой модели за счёт исправления ошибок предыдущего этапа.

Для реализации бустинга необходимо принять два важных решения:

- Метод последовательной подгонки.
- Метод агрегации.

Существуют два основных алгоритма бустинга — адаптивный бустинг и градиентный бустинг [9, 12].

Хотя бустинг может применяться к любому базовому алгоритму машинного обучения, для примера будет использован в качестве базового оценщика крайне популярный вариант — дерево решений.

Основным ограничением деревьев решений является их высокая дисперсия, что делает их склонными к переобучению. Они хорошо строят последовательные правила, но из-за этого запоминают обучающие данные и плохо обобщаются на новые данные. Необходимо рассмотреть, как преодолеть эти ограничения при использовании деревьев решений для бустинга.

Адаптивный бустинг (AdaBoost) — это последовательный ансамблевый метод, применимый как для классификации, так и для регрессии. Он может использовать любые базовые модели машинного обучения, но чаще всего применяется с деревьями решений.

В AdaBoost метод последовательной подгонки реализуется путём обновления весов каждого наблюдения в обучающем наборе данных при переходе от одной базовой модели к следующей. Метод агрегации представляет собой взвешенную сумму базовых моделей, где вес модели зависит от ошибки данного конкретного оценщика.

Обучение модели AdaBoost — это процесс определения весов наблюдений обучающего набора на каждом шаге, а также итоговых весов каждой базовой модели для агрегации.

Градиентный бустинг — это последовательный ансамблевый метод, применимый как для классификации, так и для регрессии. Он может использовать любые базовые модели машинного обучения, но чаще всего применяется с деревьями решений; такие модели называются градиентно бустлируемыми деревьями.

В градиентном бустинге метод последовательной подгонки реализуется путём подгонки базовой модели к отрицательному градиенту ошибки на предыдущем этапе. Метод агрегации — это взвешенная сумма базовых моделей, где вес модели является константой.

Обучение модели градиентного бустинга — это процесс определения ошибки базовой модели на каждом шаге и использования этих ошибок для наилучшего построения последующей базовой модели.

Первый шаг — подгонка первого базового оценщика (базовой модели). Базовые оценщики в алгоритмах бустинга, как правило, просты и имеют высокое смещение. В отличие от AdaBoost, где используются простейшие деревья решений — пни (stumps) всего с одним уровнем, — градиентно бустлируемые деревья могут и обычно содержат несколько ветвей решений. Часто такие деревья имеют до 32 листьев, что соответствует глубине дерева в 5 уровней.

После того как первая базовая модель обучена, определяются остаточные ошибки (h_1) модели на обучающих данных. Остаточная ошибка — это разность между фактическими и предсказанными значениями для каждого экземпляра обучающих данных.

$$h_1 = y_{\text{фактический}} - y_{1(\text{предсказанный})}$$

Ошибки будут больше для тех экземпляров обучающих данных, где модель предсказала хуже, и меньше там, где модель подошла к данным хорошо.

На следующем этапе последовательного обучения подгоняется вторая базовая модель. Здесь и происходит самое интересное. Вместо подгонки модели к целевым значениям $y_{\text{фактический}}$ (как это обычно делается в машинном обучении), необходимо подогнать модель к ошибкам предыдущего этапа, в данном случае к h_1 . Вторая базовая модель буквально учится на ошибках первой через вычисленные остатки.

Результаты второй базовой модели умножаются на константный параметр скорости обучения альфа и добавляются к результатам первой модели, давая обновлённые предсказания. Результаты второй модели, задачей которой была подгонка ошибок первой, умножаются на константу альфа и суммируются с результатами первой модели, что даёт обновлённые предсказания $y_{2(\text{предсказанный})}$.

Остаточные ошибки второго этапа вычисляются с использованием обновлённых предсказаний:

$$h_2 = y_{\text{фактический}} - y_{2(\text{предсказанный})}$$

Последующие этапы повторяют те же шаги. На этапе N базовая модель подгоняется к ошибкам, вычисленным на предыдущем этапе h_{N-1} . Новая подогнанная модель умножается на константный параметр скорости обучения альфа и добавляется к предсказаниям предыдущего этапа.

Когда достигнуто заданное число оценщиков в модели градиентного бустинга или остаточные ошибки перестают изменяться между итерациями, обучение останавливается, и получается итоговая ансамблевая модель. Подробное изложение ансамблевых методов приведено в [10].

ГЛАВА 2. ПРАКТИЧЕСКОЕ ПРИМЕНЕНИЕ СРЕДСТВ МАШИННОГО ОБУЧЕНИЯ ДЛЯ ПРОГНОЗИРОВАНИЯ СТРАХОВЫХ ВЫПЛАТ

2.1. Анализ исходного датасета. Предобработка данных, кодирование признаков и извлечение признаков. Первичный разведочный анализ данных (EDA). Удаление выбросов.

В рамках данной работы решается задача множественной линейной регрессии, где целевой переменной (зависимой переменной) является столбец `charges` (размер страховых выплат), а признаками (независимыми переменными) — все остальные столбцы в наборе данных [13]: `age` (возраст), `sex` (пол), `bmi` (индекс массы тела), `children` (количество детей), `smoker` (курящий — да/нет), `region` (регион). Исходные данные (датасет) взяты из открытого источника [13] и содержат 1338 записей о клиентах страховой компании.

Категориальные признаки (такие как `sex`, `smoker`, `region`, а также производный признак `weight_category`) перед обучением модели подвергаются процедуре “One-Hot Encoding”.

One-hot encoding (“прямое кодирование”) представляет собой метод преобразования категориальных переменных, при котором для каждого уникального значения категориального признака создается отдельная бинарная (фиктивная) переменная, принимающая значения 1 или 0, где 1 указывает на наличие данной категории, а 0 — на ее отсутствие.

Процесс решения задачи включает в себя следующие этапы: первичный разведочный анализ данных (EDA), предобработка данных (обработка пропусков, кодирование категориальных переменных, нормализация числовых признаков), разделение данных на обучающую и тестовую выборки, обучение,

Commented [U9]: вы уже описываете набор данных, но нигде его не упомянули, и не дали на него ссылку.

регуляризацию, оптимизацию гиперпараметров и валидацию моделей, а также сравнительный анализ их эффективности по метрикам MSE, MAE и R^2 .

MSE — это среднее значение квадратов разностей между фактическими и предсказанными значениями. Чем меньше MSE, тем лучше модель.

MAE — это среднее абсолютных значений разностей между фактическими и предсказанными значениями. Чем меньше MAE, тем лучше модель.

R^2 показывает, какую долю дисперсии целевой переменной объясняет модель. Значения варьируются от 0 до 1 (или от 0% до 100%). Чем ближе R^2 к 1, тем лучше модель.

Исходные данные выглядели следующим образом:

Таблица 2.1. Первые 5 записей исходного датасета

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Анализируя исходные данные, легко выявить дальнейшие трудности, с которыми придется столкнуться. В частности:

- Столбец `smoker` (курящий/курящая) — является категориальным признаком, что абсолютно не подходит для обучения модели при помощи алгоритма Регрессии.
- Столбец `bmi` (индекс массы тела) — не имеет линейного отношения со столбцом `charges` (выплаты). К примеру, слишком большой `bmi` (ожирение) будет негативно влиять на предсказание `charges` (выплаты), а слишком маленький `bmi` (анорексия) будет так же негативно влиять на

предсказание `charges`. Тем временем, индекс массы тела в норме будет влиять на меньший размер страховых выплат. В связи с этим у модели возникнут трудности при обучении.

- Столбец `sex` (пол) — является категориальным признаком, тот же самый случай что и со столбцом `smoker`.
- Столбец `region` (регион проживания) — так же является категориальным признаком, с 4 допустимыми значениями.

Поскольку столбцы `smoker` и `sex` имеют лишь два допустимых значения, было принято решение заменить следующие значения:

- Мужской (пол): 1
- Женский (пол): 0
- Да (курящий/курящая): 1
- Нет (курящий/курящая): 0

Столбец `bmi` был подвергнут процедуре “One-Hot Encoding” (прямое кодирование). Вследствие чего стал признаком `weight_category` (весовая категория), со следующими допустимыми значениями:

- Obesity (ожирение): если $bmi > 30.0$
- Overweight (избыточный вес): если $bmi \geq 25.0$ и $bmi \leq 30.0$
- Normal weight (норма): если $bmi \geq 18.5$ и $bmi < 25.0$
- Underweight (недостаток веса): если $bmi < 18.5$

Столбец `region` так же был подвергнут процедуре “One-Hot Encoding”, однако здесь не потребовались никакие сложные вычисления. Достаточно было внести этот признак в аргумент функции `pd.get_dummies` — `columns`, который является массивом из строк-названий признаков.

Финальным шагом предобработки признаков станет удаление следующих признаков из тренировочного датафрейма:

- `bmi` (индекс массы тела)
- `weight_category_Normal` `weight` (произвольный признак категория веса ‘норма’)

- region_southwest (произвольный признак регион ‘юго-запад’)

bmi был удален за отсутствием дальнейшей необходимости, а weight_category_Normal weight и region_southwest были удалены, так как при применении процедуры ‘One-Hot Encoding’ для обучения линейной регрессии следует всегда исключать одну переменную, чтобы избежать ловушки фиктивных переменных.

Таблица 2.2. Суммирование данных исходного датасета

Commented [U10]: оформите таблицы

	age	sex	bmi	children	smoker	region	charges
count	1338.000000	1338	1338.000000	1338.000000	1338	1338	1338.000000
unique	NaN	2	NaN	NaN	2	4	NaN
top	NaN	male	NaN	NaN	no	southeast	NaN
freq	NaN	676	NaN	NaN	1064	364	NaN
mean	39.207025	NaN	30.663397	1.094918	NaN	NaN	13270.422265
std	14.049960	NaN	6.098187	1.205493	NaN	NaN	12110.011237
min	18.000000	NaN	15.960000	0.000000	NaN	NaN	1121.873900
25%	27.000000	NaN	26.296250	0.000000	NaN	NaN	4740.287150
50%	39.000000	NaN	30.400000	1.000000	NaN	NaN	9382.033000
75%	51.000000	NaN	34.693750	2.000000	NaN	NaN	16639.912515
max	64.000000	NaN	53.130000	5.000000	NaN	NaN	63770.428010

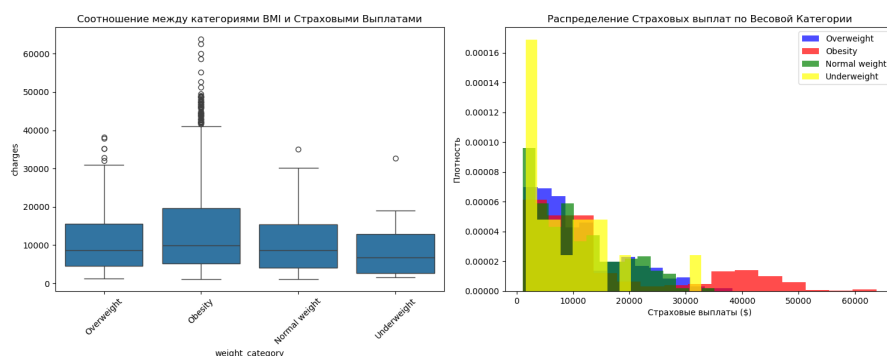


Рис. 2.3. Анализ соотношения признаков bmi (weight_category) и charges. Просмотр выбросов

Из первого графика видно, что у людей с bmi (индексом массы тела), классифицируемым как ожирение, как правило, более высокие страховые выплаты. Нормальный вес и избыточный вес в целом одинаковы. Выплаты с недостаточным весом — немного ниже. Так же, отчетливо заметно что в датасете присутствуют выбросы.

Необходимо так же посмотреть на признак charges (страховые выплаты) в отдельности.

- Среднее значение: 13270.422265141257
- Медиана: 9382.033
- Мода: 0 1639.5631
- Усеченное среднее значение: 9877.310386652985

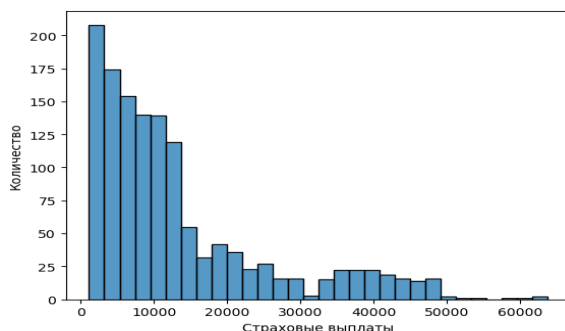


Рис. 2.4. Разброс страховых выплат

Из графика видно, что данные имеют выраженное правостороннее смещение (положительную асимметрию), что приводит к тому, что среднее значение оказывается выше медианы. Это указывает на наличие экстремально высоких значений в правой части распределения. Так же видно, что усеченное среднее значение достаточно близко к медиане и моде. Особенно это заметно при отсечении 20% данных с каждой стороны распределения. На основе анализа можно сделать вывод, что типичная стоимость страховки составляет около 9400 долларов.

Так же, разумным станет посчитать дисперсию charges (размера страховых выплат):

- Размах: 62648.554110000005
- Межквартильный размах: 11899.625365
- Дисперсия: 146652372.15285483
- Стандартное отклонение: 12110.011236693996
- Среднее абсолютное отклонение: 9091.12658113703

Из этих данных видно, что размах (Range) не подходит для описания данных в данном случае, поскольку у одного или нескольких человек стоимость страховых выплат превышает 60 000\$. Следовательно, следует рассмотреть межквартильный размах (IQR). Стандартное отклонение не является достаточно информативной мерой в данном случае, поскольку данные имеют выраженное

правостороннее смещение. Поэтому целесообразно рассмотреть среднее абсолютное отклонение (Mean Absolute Deviation). Стало заметно, что центральные 50% данных сконцентрированы вокруг медианы, а также, что страховые затраты большинства людей тесно сгруппированы вокруг типичной стоимости.

В связи с выраженным правосторонним смещением, необходимо принять решение об удалении выбросов в датасете. Выбранным методом станет IQR (межквартильный размах). IQR — это разница между значениями 75-го и 25-го перцентилей.

```
Q1 = y.quantile(0.25)
Q3 = y.quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

mask = (y >= lower) & (y <= upper)
X = X[mask]
y = y[mask]
```

В результате чего было удалено 139 выбросов. Осталось 1199 образцов.

Процентное соотношение между курящими и не курящими

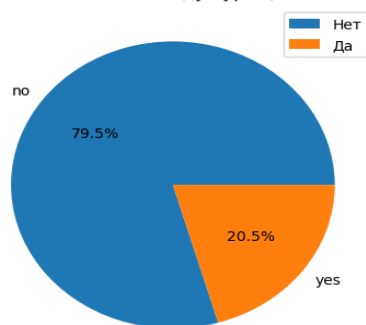


Рис. 2.5. Круговая диаграмма признака smoker (курящий/курящая)

Из круговой диаграммы отчетливо заметно, что большинство людей в датасете – не курящие.

Таблица 2.6. Сравнение среднего значения размера страховых выплат и smoker (курящий/курящая)

	charges	smoker
0	16884.92400	yes
1	1725.55230	no
2	4449.46200	no
3	21984.47061	no
4	3866.85520	no

- Разница между средним значением размера страховых выплат и средним значением признака smoker: 23615.963533676637
- Разница между медианой размера страховых выплат и медианой признака smoker: 27110.943150000006

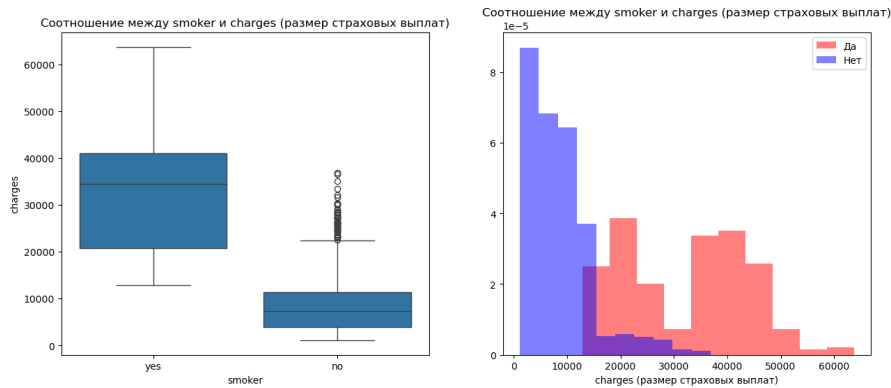


Рис. 2.7. Соотношение между smoker и размером страховых выплат

Разница является слишком значительной, следовательно, можно предположить, что курение существенно увеличивает стоимость страховки. Оба графика показывают, что между этими категориями отсутствует перекрытие,

поэтому можно сделать вывод, что курение значительно увеличивает стоимость страховых выплат.

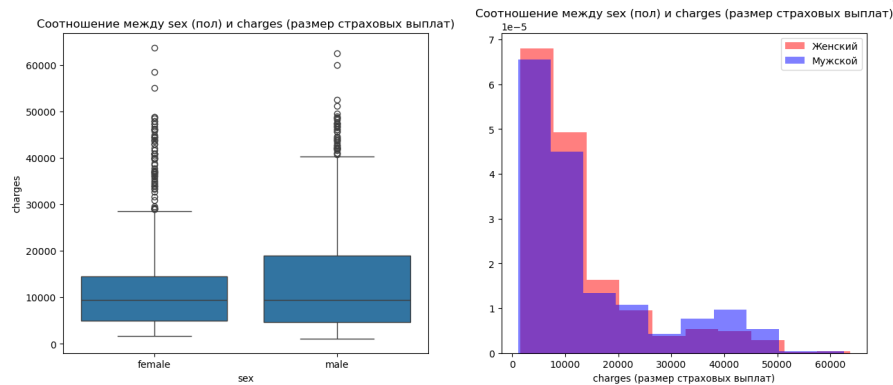


Рис. 2.8. Соотношение между sex (пол) и charges (размером страховых выплат)

- Разница между средним значением страховых выплат у мужчин и женщин: 1387.1723338865468
- Разница между медианой страховых выплат у мужчин и женщин: 43.34674999999879

Разница между средними значениями и медианами, а так же графики, показывают что между мужчинами и женщинами не существует явной разницы в стоимости страхования.

2.2. Обоснование выбора моделей машинного обучения. Анализ результатов обучения. Описание метрик MAE, RMSE, R².

Для решения задачи прогнозирования страховых выплат были выбраны три типа моделей, различающихся по сложности и способности улавливать нелинейные зависимости.

Линейная регрессия (включая ридж и лассо) использовалась в качестве базовой модели. Она проста для интерпретации и позволяет количественно оценить линейный вклад каждого признака. Регуляризация (L2 и L1) добавлялась для борьбы с переобучением.

Случайный лес (Random Forest) выбран как ансамблевый метод, автоматически учитывающий нелинейные взаимодействия между признаками, устойчивый к выбросам и не требующий масштабирования данных. Он также даёт оценку важности признаков.

Градиентный бустинг (Gradient Boosting) выбран потому, что он часто показывает наилучшую точность на табличных данных. Его последовательная коррекция ошибок позволяет модели «учить» сложные закономерности, что особенно важно для предсказания редких, но дорогих страховых случаев.

Для анализа результатов были выбраны следующие метрики:

MAE (Mean Absolute Error) — средняя абсолютная ошибка. MAE измеряет среднюю величину ошибки модели в тех же единицах, что и целевая переменная (в долларах). Она вычисляется как среднее арифметическое абсолютных разностей между фактическими и предсказанными значениями:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Преимущество MAE в том, что она легко интерпретируется: «в среднем модель ошибается на X долларов». Недостаток — она одинаково учитывает как маленькие, так и большие ошибки, то есть не штрафует сильнее за крупные промахи.

RMSE (Root Mean Squared Error) — среднеквадратичная ошибка. RMSE также измеряет ошибку в единицах целевой переменной, но придаёт больший вес крупным ошибкам за счёт возведения разности в квадрат перед извлечением корня:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Это свойство делает RMSE особенно полезной, когда большие ошибки нежелательны (например, при прогнозировании страховых выплат — ошибка в 10 000 долларов гораздо критичнее, чем ошибка в 1000 долларов). Если RMSE значительно больше MAE, это указывает на наличие крупных выбросов или сильных отклонений в предсказаниях.

R² (коэффициент детерминации) — R² показывает, какую долю дисперсии (изменчивости) целевой переменной объясняет построенная модель. Формула:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Где $\bar{y}$ — среднее значение целевой переменной. R² принимает значения от 0 до 1 (иногда может быть отрицательным, если модель предсказывает хуже, чем просто среднее). Чем ближе R² к 1, тем лучше модель объясняет данные. Например, R² = 0.609 означает, что модель объясняет 60.9% изменчивости страховых выплат, а остальные 39.1% приходятся на неучтённые факторы или случайный шум.

Таблица 2.9. Результаты обучения

Модель	MAE (\$)	RMSE (\$)	R ²
Линейная регрессия	2815	5255	0.557
Ридж регрессия	2807	5248	0.559
Лассо регрессия	2796	5248	0.558
Случайный лес	2642	5133	0.578
Градиентный бустинг	2498	4938	0.609

Лучшая модель: Градиентный бустинг ($R^2 = 0.609$, MAE = 2498\$)

- Линейная регрессия ($R^2 \approx 0.557$) показала наименьшую точность. Это объясняется тем, что реальная зависимость между признаками (возраст, индекс массы тела, курение) и стоимостью страховки нелинейна. Например, модель дает разные средние предсказания для каждой весовой категории.: риск высок как при недостатке веса, так и при ожирении.

- Ридж и Лассо ($R^2 \approx 0.558$) практически не улучшили результат, поскольку добавление регуляризации не решает проблему нелинейности, а лишь борется с переобучением. Однако подбор гиперпараметров показал, что оптимальная сила регуляризации невелика (альфа = 1 для Ридж, альфа = 10 для Лассо), что говорит о том, что исходные признаки уже достаточно информативны.

- Случайный лес ($R^2 \approx 0.578$) превзошёл линейные модели, так как он автоматически моделирует нелинейные взаимодействия (например, курение $\times$ возраст) и менее чувствителен к выбросам. Однако его точность оказалась лишь незначительно выше, возможно, из-за того, что после удаления выбросов данные стали более однородными.

- Градиентный бустинг ($R^2 \approx 0.609$) показал наилучший результат. Он последовательно исправляет ошибки предыдущих деревьев, уделяя больше внимания “трудным” для предсказания наблюдениям. Это особенно важно для страховых выплат, где распределение имеет длинный правый хвост. Наилучшие параметры: `learning_rate = 0.1`, `max_depth = 3`, `n_estimators = 50`. Небольшая глубина деревьев (3) предотвращает переобучение, а количество деревьев 50 достаточно для сходимости.

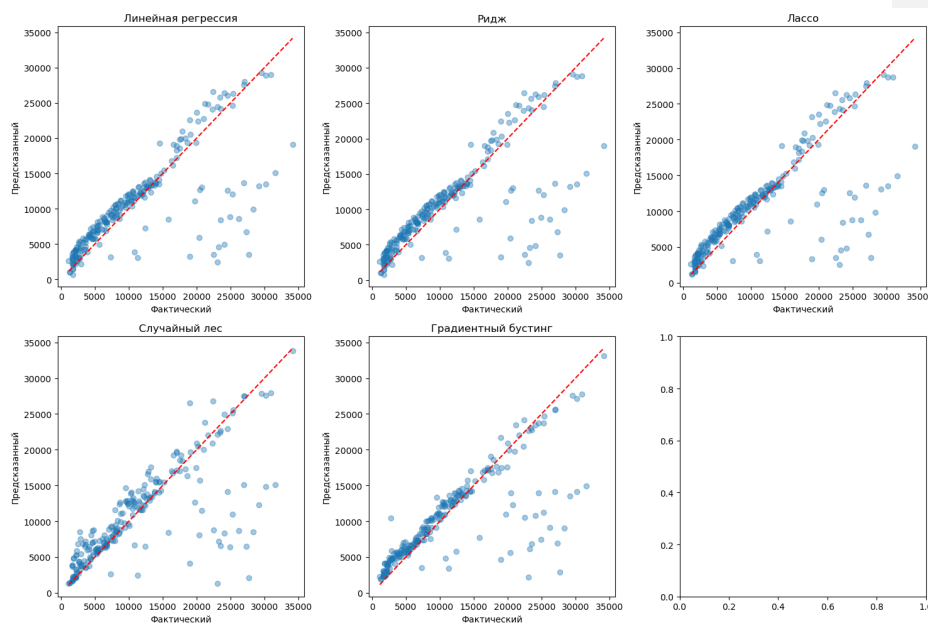


Рис. 2.10. Сравнение моделей

2.3. Вывод. Выбор лучшей модели

Лучшей моделью признан градиентный бустинг (Gradient Boosting Regressor). Он достиг наивысшего значения коэффициента детерминации ($R^2 = 0.609$), минимальной средней абсолютной ошибки ($MAE = 2498$) и минимальной среднеквадратичной ошибки ($RMSE = 4938$).

Выбранная модель (градиентный бустинг) имеет $MAE = 2498$, то есть в среднем ошибается на 2500 долларов. $RMSE = 4938$ — что выше MAE , это говорит о наличии отдельных предсказаний с крупной ошибкой. $R^2 = 0.609$ означает, что модель объясняет чуть более 60% изменчивости страховых выплат, что является удовлетворительным результатом для задачи с большим количеством неучтённых факторов (образ жизни, наследственность, хронические заболевания)

Причины превосходства градиентного бустинга:

1. Способность моделировать сложные нелинейные зависимости и взаимодействия признаков.
2. Последовательная коррекция ошибок, что особенно полезно для предсказания дорогих страховых случаев.
3. Устойчивость к выбросам (особенно при небольшой глубине деревьев).

Таким образом, градиентный бустинг рекомендуется к использованию для прогнозирования страховых выплат в реальных задачах.

Список используемых источников

1. Воронцов К. В. Машинное обучение : курс лекций / К. В. Воронцов. — Москва : МФТИ, 2019. — 214 с. — ISBN 978-5-7417-0711-8.
2. Нилд Т. Математика для Data Science : основы математики и статистики для анализа данных / Т. Нилд ; пер. с англ. — Москва : ДМК Пресс, 2020. — 256 с. — ISBN 978-5-97060-847-6.
3. Волкова С. С. Введение в машинное обучение. Линейные модели : учебное пособие / С. С. Волкова. — Новосибирск : НГТУ, 2021. — 108 с. — ISBN 978-5-7782-4298-6.
4. Горбаченко В. И. Машинное обучение : учебник для вузов / В. И. Горбаченко. — Москва : Юрайт, 2022. — 387 с. — ISBN 978-5-534-14023-5.
5. Груздев А. Прогнозное моделирование на Python с помощью деревьев решений и случайного леса / А. Груздев. — Москва : ДМК Пресс, 2020. — 220 с. — ISBN 978-5-97060-830-8.
6. Митина О. А. Технологии и инструментарий машинного обучения : учебное пособие / О. А. Митина, В. В. Жаров. — Томск : Изд-во Томского политехнического университета, 2019. — 149 с. — ISBN 978-5-4387-0821-5.
7. Ефимов А. И. Основы машинного обучения : учебное пособие / А. И. Ефимов. — Санкт-Петербург : Университет ИТМО, 2021. — 112 с. — Режим доступа: <https://books.ifmo.ru/file/pdf/2844.pdf> (дата обращения: 11.06.2026).
8. Князюк Н. В. Методы машинного обучения на Python : учебное пособие / Н. В. Князюк, О. В. Рыкова. — Иркутск : ИГУ, 2026. — 180 с. — ISBN 978-5-9624-2067-6.
9. Friedman J. H. Greedy Function Approximation: A Gradient Boosting Machine / J. H. Friedman // The Annals of Statistics. — 2001. — Vol. 29, № 5. — P. 1189–1232. — DOI: 10.1214/aos/1013203451.
10. Hastie T. The Elements of Statistical Learning : Data Mining, Inference, and Prediction / T. Hastie, R. Tibshirani, J. Friedman. — 2nd ed. — New York :

Commented [U11]: н е т, онлайн курсы не являются научными источниками. Это могут быть статьи, монографии, учебные пособия. В крайнем случае сайты с аналитикой и ГОСТы. Вы можете оставить 1-2 подобные ссылки, но не более.

Commented [U12]: н е т, на русском языке тоже должны быть источники обязательно.

Springer, 2009. — 745 p. — ISBN 978-0-387-84857-0. — DOI: 10.1007/978-0-387-84858-7.

11. Scikit-learn: Machine Learning in Python [Электронный ресурс]
Режим доступа: <https://scikit-learn.org/stable/> (дата обращения: 11.06.2026).

12. Scikit-learn: Machine Learning in Python. Gradient Boosting Regressor [Электронный ресурс]. — Режим доступа: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html> (дата обращения: 13.05.2026).

13. Lopes, M. Medical Cost Personal Datasets [Электронный ресурс]. — Режим доступа: <https://www.kaggle.com/datasets/mirichoi0218/insurance> (дата обращения: 21.05.2026).

14. Billa M. M. Medical Insurance Price Prediction Using Machine Learning / M. M. Billa, T. Nagpal // Journal of Engineering Sciences. — 2024. — Vol. 15, № 3. — P. 120–128. — Режим доступа: <https://journal.esrgroups.org/jes/article/view/3962> (дата обращения: 11.06.2026).

15. Machine Learning in Insurance: Enhancing Pricing, Claim Detection, and Index Insurance Innovation // Proceedings of the 18th International Conference on Agents and Artificial Intelligence (ICAART). — 2024. — P. 127–135. — Режим доступа: <https://www.scitepress.org/Papers/2024/135300/135300.pdf> (дата обращения: 11.05.2026).

Приложения

Commented [U13]: Весь программный код оформите одним приложением. Можете озаглавить разные файлы, но не выносите их заголовки в оглавление

Приложение 1.

```
import pandas as pd

med_insurance = pd.read_csv('./data/insurance.csv')
print(med_insurance.head(5))

partition_by_bmi = lambda bmi: 'Underweight' if bmi < 25.0 else 'Overweight'

bmi_copy = med_insurance['bmi']
med_insurance_bmi = bmi_copy.apply(partition_by_bmi)
med_insurance_bmi = med_insurance_bmi.to_frame()

med_insurance_bmi.rename(columns={'bmi': 'weight_category'}, inplace=True)
med_insurance_bmi['bmi'] = med_insurance.bmi

partition_by_bmi_2 = lambda row: 'Normal weight' if (row.bmi >= 18.5 and
row.bmi < 25.0) else row.weight_category
med_insurance_bmi['weight_category'] =
med_insurance_bmi.apply(partition_by_bmi_2, axis=1)

partition_by_bmi_3 = lambda row: 'Obesity' if row.bmi > 30.0 else
row.weight_category
med_insurance_bmi['weight_category'] =
med_insurance_bmi.apply(partition_by_bmi_3, axis=1)

med_insurance_2 = med_insurance[['age', 'sex', 'bmi', 'children', 'smoker',
'region', 'charges']]
med_insurance_2['weight_category'] = med_insurance_bmi['weight_category']
med_insurance_2 = pd.get_dummies(data=med_insurance_2,
columns=['weight_category', 'region'], dtype='int')

med_insurance_2.to_csv('./data/med_insurance_processed.csv', index=False)

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
```

```

import matplotlib.pyplot as plt

df = pd.read_csv('./data/med_insurance_processed.csv')

df['smoker'] = df.smoker.apply(lambda smoker: 1 if smoker == 'yes' else 0)
df['sex'] = df.sex.apply(lambda sex: 1 if sex == 'male' else 0)
df = df.drop(columns=['bmi', 'weight_category_Normal weight',
'region_southwest'])

print(df.head(5))

X = df.drop(columns='charges')
y = df['charges']

Q1 = y.quantile(0.25)
Q3 = y.quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

mask = (y >= lower) & (y <= upper)
X = X[mask]
y = y[mask]
num_cols = ['age', 'children']
scaler = StandardScaler()

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

ridge = Ridge()
param_grid_ridge = {'alpha': [0.01, 0.1, 1, 10, 100]}
grid_ridge = GridSearchCV(ridge, param_grid_ridge, cv=5, scoring='r2')
grid_ridge.fit(X_train, y_train)
best_ridge = grid_ridge.best_estimator_
y_pred_ridge = best_ridge.predict(X_test)

lasso = Lasso(max_iter=10000)
param_grid_lasso = {'alpha': [0.001, 0.01, 0.1, 1, 10]}
grid_lasso = GridSearchCV(lasso, param_grid_lasso, cv=5, scoring='r2')

```

```

grid_lasso.fit(X_train, y_train)
best_lasso = grid_lasso.best_estimator_
y_pred_lasso = best_lasso.predict(X_test)

print(f"Best Ridge alpha: {grid_ridge.best_params_['alpha']}")
print(f"Best Lasso alpha: {grid_lasso.best_params_['alpha']}")

rf = RandomForestRegressor(random_state=42)
param_grid_rf = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10]
}
grid_rf = GridSearchCV(rf, param_grid_rf, cv=5, scoring='r2', n_jobs=-1)
grid_rf.fit(X_train, y_train)
best_rf = grid_rf.best_estimator_
y_pred_rf = best_rf.predict(X_test)

print(f"Best RF params: {grid_rf.best_params_}")

gb = GradientBoostingRegressor(random_state=42)
param_grid_gb = {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7]
}
grid_gb = GridSearchCV(gb, param_grid_gb, cv=5, scoring='r2', n_jobs=-1)
grid_gb.fit(X_train, y_train)
best_gb = grid_gb.best_estimator_
y_pred_gb = best_gb.predict(X_test)

print(f"Best GB params: {grid_gb.best_params_}")

def evaluate(y_true, y_pred, model_name):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)
    print(f"{model_name:20} | MAE: ${mae:8.2f} | RMSE: ${rmse:8.2f} | R²: {r2:.4f}")
    return {'model': model_name, 'MAE': mae, 'RMSE': rmse, 'R2': r2}

results = []
results.append(evaluate(y_test, y_pred_lr, "Linear Regression"))
results.append(evaluate(y_test, y_pred_ridge, "Ridge Regression"))
results.append(evaluate(y_test, y_pred_lasso, "Lasso Regression"))

```

```

results.append(evaluate(y_test, y_pred_rf, "Random Forest"))
results.append(evaluate(y_test, y_pred_gb, "Gradient Boosting"))

results_df = pd.DataFrame(results)
print("\nSummary:")
print(results_df.to_string(index=False))
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
models = [('Линейная регрессия', y_pred_lr), ('Ридж', y_pred_ridge),
          ('Лассо', y_pred_lasso),
          ('Случайный лес', y_pred_rf), ('Градиентный бустинг', y_pred_gb)]
axes = axes.flatten()
for i, (name, pred) in enumerate(models):
    axes[i].scatter(y_test, pred, alpha=0.4)
    axes[i].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
                 'r--')
    axes[i].set_xlabel('Фактический')
    axes[i].set_ylabel('Предсказанный')
    axes[i].set_title(name)
plt.tight_layout()
plt.show()
charges_mean = med_insurance.charges.mean()
print("Mean:", charges_mean)

charges_median = med_insurance.charges.median()
print("Median:", charges_median)

charges_mode = med_insurance.charges.mode()
print("Mode:", charges_mode)

from scipy.stats import trim_mean
charges_trim_mean = trim_mean(med_insurance.charges, proportiontocut=0.2)
print("Trimmed mean:", charges_trim_mean)
print("\n")

sns.histplot(x='charges', data=med_insurance)
plt.xlabel("Страховые выплаты")
plt.ylabel("Количество")
plt.show()
plt.close()

charges_range = med_insurance.charges.max() - med_insurance.charges.min()
print("Range:", charges_range)

charges_iqr = med_insurance.charges.quantile(0.75) -
med_insurance.charges.quantile(0.25)
print("Interquartile range:", charges_iqr)

```

```

charges_variance = med_insurance.charges.var()
print("Variance:", charges_variance)

charges_std = med_insurance.charges.std()
print("Standard deviation:", charges_std)

def mad(series):
    return (series - series.mean()).abs().mean()

charges_mad = mad(med_insurance.charges)
print("Mean Absolute Deviation:", charges_mad)
print("\n")

med_insurance.smoker.value_counts()

med_insurance.smoker.value_counts().plot.pie(autopct='%1.1f%%')
plt.title('Процентное соотношение между курящими и не курящими')
plt.ylabel(' ')
plt.legend(['Нет', 'Да'])
plt.show()
plt.close()

insurance_smoker_charges = med_insurance[['charges', 'smoker']]
print(insurance_smoker_charges.head(5))

insurance_smoker =
insurance_smoker_charges.charges[insurance_smoker_charges.smoker == 'yes']
insurance_non_smoker =
insurance_smoker_charges.charges[insurance_smoker_charges.smoker == 'no']

insurance_smoker_mean = insurance_smoker.mean()
insurance_non_smoker_mean = insurance_non_smoker.mean()

insurance_smoker_median = insurance_smoker.median()
insurance_non_smoker_median = insurance_non_smoker.median()

print("The difference between charges mean and smoker mean:",
insurance_smoker_mean - insurance_non_smoker_mean)
print("The difference between charges median and smoker median:",
insurance_smoker_median - insurance_non_smoker_median)
print("\n")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

```

```

sns.boxplot(data = insurance_smoker_charges, x = 'smoker', y = 'charges',
ax=ax1)
ax1.set_title("Соотношение между smoker и charges (размер страховых
выплат)")

ax2.hist(insurance_smoker , color="red", label="yes", density=True,
alpha=0.5)
ax2.hist(insurance_non_smoker , color="blue", label="no", density=True,
alpha=0.5)
ax2.legend(['Да', 'Нет'])
plt.xlabel('charges (размер страховых выплат)')
ax2.set_title("Соотношение между smoker и charges (размер страховых
выплат)")

plt.show()

insurance_sex_charges = med_insurance[['charges', 'sex']]
print(insurance_smoker_charges.head(5))

insurance_male = insurance_sex_charges.charges[insurance_sex_charges.sex ==
'male']
insurance_female = insurance_sex_charges.charges[insurance_sex_charges.sex
== 'female']

insurance_male_mean = insurance_male.mean()
insurance_female_mean = insurance_female.mean()

insurance_male_median = insurance_male.median()
insurance_female_median = insurance_female.median()

print("The difference between male mean and female mean:",
insurance_male_mean - insurance_female_mean)
print("The difference between male median and female median:",
insurance_male_median - insurance_female_median)
print("\n")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
sns.boxplot(data = insurance_sex_charges, x = 'sex', y = 'charges', ax=ax1)
ax1.set_title("Соотношение между sex (пол) и charges (размер страховых
выплат)")

ax2.hist(insurance_female , color="red", label="female", density=True,
alpha=0.5)
ax2.hist(insurance_male , color="blue", label="male", density=True,
alpha=0.5)
ax2.legend(['Женский', 'Мужской'])

```

```
plt.xlabel('charges (размер страховых выплат)')  
ax2.set_title("Соотношение между sex (пол) и charges (размер страховых  
выплат)")  
plt.show()
```

Приложение 2.

Глоссарий

Термин	Определение
Регрессия	Задача машинного обучения с учителем, в которой предсказывается непрерывная числовая величина.
Регуляризация	Метод добавления штрафного члена к функции потерь для предотвращения переобучения.
Градиентный бустинг	Последовательный ансамблевый метод, в котором каждая новая модель корректирует ошибки предыдущей.
Бэггинг (Bagging)	Агрегирование бутстреп-выборок — метод построения ансамбля путём обучения базовых моделей на разных случайных подмножествах данных.
Гиперпараметр	Параметр модели, который не обучается по данным, а задаётся до начала обучения (например, глубина дерева, скорость обучения).
Бутстреп-выборка	Метод случайной выборки с возвращением из исходного набора данных, используемый в бэггинге для создания разных обучающих подмножеств.